

6. PROBLEMAS DE SATISFACCIÓN DE RESTRICCIONES

(PARTE 2)

IA 3.2 - Programación III

1º C - 2026

Dr. Mauro Lucci



Próximamente

— — —

Los algoritmos de búsqueda clásica se pueden mejorar incorporando **heurísticas** que requieren de información derivada de nuestro conocimiento del problema específico a resolver.

Backtracking se puede mejorar **independientemente del problema.**

¿Cómo es posible? 🤔

Backtracking mejorado

Backtracking mejorado

— — —

- Backtracking requiere **critérios** para elegir una variable no asignada y para ordenar los valores de su dominio.

```
1 function BACKTRACK(csp, asign) return solución o fallo
...
3    $X_i \leftarrow$  ELEGIR_VAR(csp, asign)
4   for each v in ORDENAR_DOM(csp, asign,  $X_i$ ) do
...
```

- Pueden ser **estáticos** o **dinámicos**.
- Buenas decisiones podrían evitar retrocesos a futuro y así acelerar el hallazgo de soluciones.

Criterios estáticos

— — —

- En los criterios **estáticos** el orden se fija antes de ejecutar el algoritmo.

📖 **Ejemplo.** Las variables se asignan en orden “alfabético”. Es decir, en la raíz se asigna la primera variable, en el nivel uno la segunda variable, y así sucesivamente.

- 👍 Simples.
- 👎 Rara vez conducen a búsquedas eficientes.

Criterios dinámicos

— — —

- En los criterios **dinámicos** el orden se decide mientras se ejecuta el algoritmo.
 - En general, toman decisiones en base a la asignación parcial del nodo actual. Veremos los siguientes criterios.
 - **Mínimos valores restantes (MVR).**
 - **Grado.**
 - **Valor menos restrictivo.**
- } Para elegir la **variable**.
- } Para elegir el **valor** de la variable.

Mínimos valores restantes

— — —

Criterio de mínimos valores restantes (MVR). La próxima variable a asignar es la que tiene menor número de valores **consistentes** en su dominio.



La idea es asignar tales variables antes de que se queden sin valores consistentes (y el algoritmo deba perder tiempo retrocediendo).



Ejemplo – Coloreo de Australia

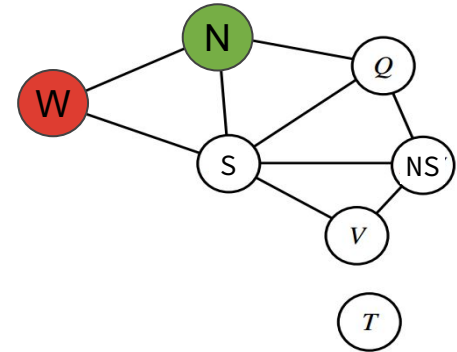
— — —

Sea la asignación parcial $\{W=\text{red}, N=\text{green}\}$.

Variable no asignada	Q	S	NS	V	T
Valores consistentes	 		  	  	  

MVR elegirá asignar la variable S.

Notar que cualquiera de las acciones (Q, blue) , (NS, blue) y (V, blue) genera que no queden valores consistentes para S.



Grado

— — —

Criterio de grado. La próxima variable a asignar es la que está relacionada con el mayor número de variables **no asignadas**.



La intuición es que tales variables están muy restringidas y corren riesgo de quedarse sin valores consistentes.

Cuando todas las variables tienen el mismo dominio, MVR genera un empate en la raíz. **El criterio de grado suele usarse para desempatar.**



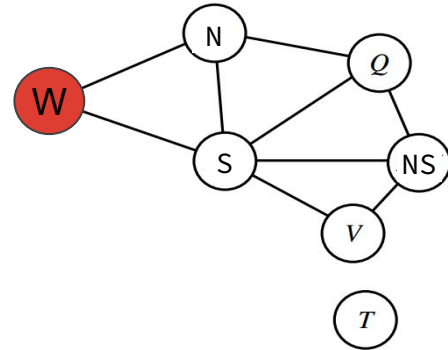
Ejemplo – Coloreo de Australia

Sea la asignación parcial $\{W=\text{red}\}$.

- S se relaciona con **cuatro** variables no asignadas: N, Q, NS y V,
- Q con **tres**: N, NS y S,
- NS con **tres**: Q, V y S,
- N con **dos**: Q y S,
- V con **dos**: NS y S,
- T con **cero**.

El criterio de grado elegirá asignar la variable S.

Asignar cualquiera de las variables N, Q, NS o V puede restringir los valores consistentes para S.





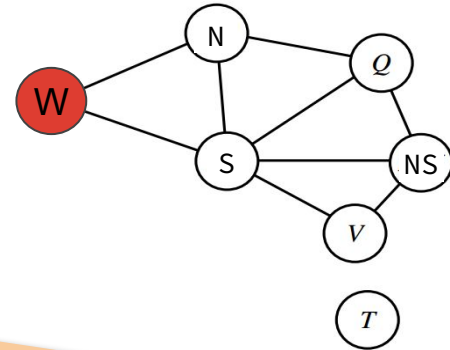
Ejemplo – Coloreo de Australia

Sea la asignación parcial $\{W=\text{red}\}$.

- S se relaciona con **cuatro** variables no asignadas: N, Q, NS y V,
- Q con **tres**: N, NS y S,
- NS con **tres**: Q, V y S,
- N con **dos**: Q y S,
- V con **dos**: NS y S,
- T con **cero**.

El criterio de grado elegirá asignar la variable S.

Asignar cualquiera de las variables N, Q, NS o V puede restringir los valores consistentes para S.



Estamos calculando el **grado** de los vértices en el subgrafo inducido por los vértices correspondientes a variables no asignadas.

Valor menos restrictivo

Elegida la variable, queda elegir un valor de su dominio.

Criterio de valor menos restrictivo. Dada una variable, el próximo valor a elegir es el que inhabilita menos valores en las demás variables no asignadas.



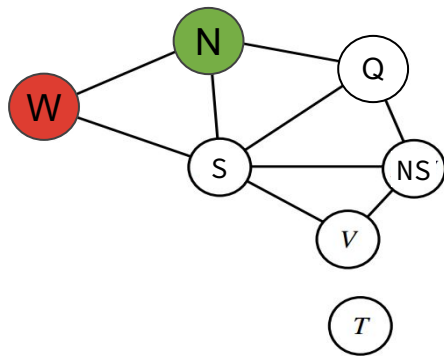
La intuición es que es más fácil encontrar soluciones cuando las variables tienen mayor número de valores consistentes en sus dominios.



Ejemplo – Coloreo de Australia

Sea la asignación parcial $\{W=\text{red}, N=\text{green}\}$, entonces:

Variable no asignada	Q	S	NS	V	T
Valores consistentes	red blue	blue	red blue green	red blue green	red blue green



Suponer que se elige a Q (por algún criterio). Hay dos valores posibles:

- (Q, blue) descarta 2 valores consistentes (blue de S y blue de NS).
- (Q, red) descarta 1 valor consistente (red de NS, notar que red ya había sido descartado de S cuando se asignó a W).

El criterio de valor menos restrictivo elige primero a red y luego a blue para Q.

Backtracking mejorado

- Con estos criterios ya podemos implementar un algoritmo de backtracking bastante digno.
 - Por ejemplo, resuelve el problema de las 1000 reinas.
- **No precisan conocimiento específico del problema** (a diferencia de las heurísticas usadas en las búsquedas informadas).
- Son **generales** (aplican a cualquier CSP).

Un poco más allá...

— — —

Existen otros mecanismos que pueden incorporarse a backtracking para ayudar a mejorar su performance.

3. **Comprobación hacia adelante.**
4. **Propagación de restricciones.**
5. **Restricciones globales.**

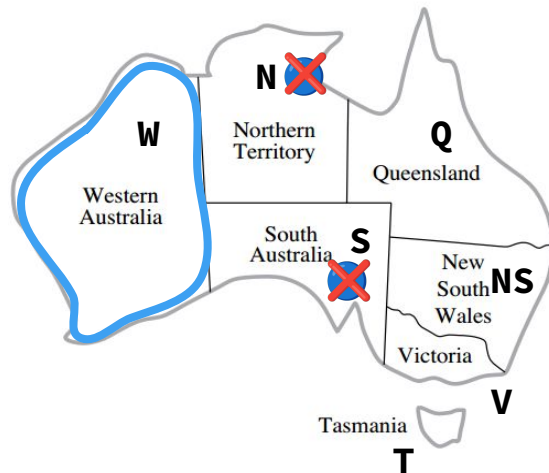
Ejemplo – Coloreo de Australia

— — —

Supongamos que W se pinta con ●.

Luego, pintar N o S también con ● llevaría a una asignación inconsistente con las restricciones $W \neq N$ y $W \neq S$.

Entonces, luego de asignar $W = \text{●}$, podemos quitar a ● de los dominios de N y S.



3. Comprobación hacia adelante

— — —

- **Comprobación hacia adelante.** Siempre que se asigne una variable X_i , se mira cada variable no asignada X_j que esté relacionada con X_i y se quita del dominio de X_j cualquier valor inconsistente con el valor asignado a X_i .
- Los dominios se van achicando a medida que se asignan variables.
- Si una variable no asignada se queda con **dominio vacío**, significa que la asignación es inconsistente y la búsqueda debe **retroceder**.



Ejemplo – Coloreo de Australia

— — —

Volvamos al ejemplo de la clase pasada, esta vez aplicando **backtracking con comprobación hacia adelante**.

Usaremos el siguiente orden estático para las variables.

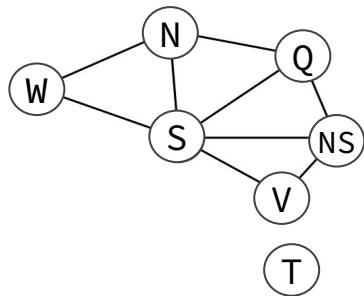
Orden	1°	2°	3°	4°	5°	6°	7°
Variable	W	N	Q	S	NS	V	T

Por cuestiones de espacio no escribimos la tabla con la frontera (de ser necesario puede verse en las diapositivas anteriores).



Ejemplo

— — —



N° it.	Dominio						
	W	N	Q	NS	V	S	T
0							
1							
2							
3							
4							
5							
6							
7							
8							

Inicialmente todas las variables tienen los 3 colores en su dominio.

n_0
{ }

Orden:

1) W

2) N

3) Q

4) S

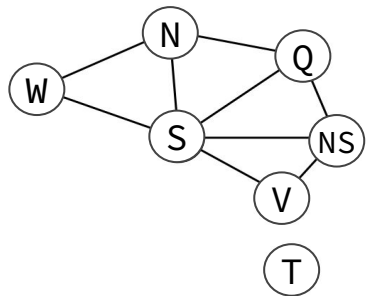
5) NS

6) V

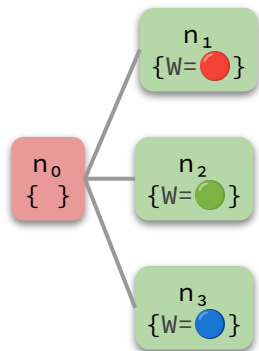
7) T



Ejemplo



N° it.	Dominio						
	W	N	Q	NS	V	S	T
0	  	  	  	  	  	  	  
1							
2							
3							
4							
5							
6							
7							
8							



Orden:

1) W

2) N

3) Q

4) S

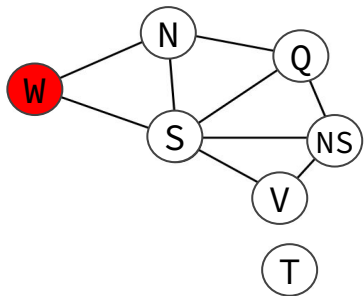
5) NS

6) V

7) T



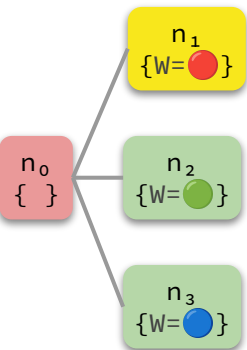
Ejemplo



N° it.	Dominio						
	W	N	Q	NS	V	S	T
0	● ● ●	● ● ●	● ● ●	● ● ●	● ● ●	● ● ●	● ● ●
1	●	● ●	● ● ●	● ● ●	● ● ●	● ●	● ● ●
2							
3							
4							
5							
6							
7							
8							

Comprobación hacia adelante.

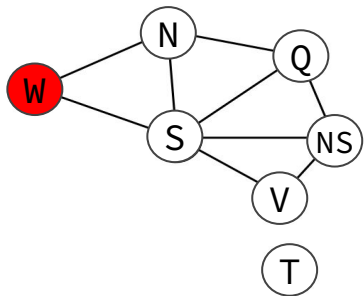
Luego de asignar $W = \text{red}$, en las variables no asignadas N y S, ambas relacionadas con W, se debe eliminar a red de su dominio. Es decir, el dominio de N y S queda $\{\text{green}, \text{blue}\}$.



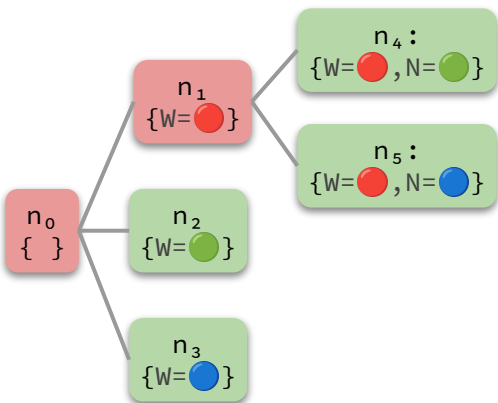
Orden: 1) W 2) N 3) Q 4) S 5) NS 6) V 7) T



Ejemplo



N° it.	Dominio							
	W	N	Q	NS	V	S	T	
0	  	  	  	  	  	  	  	
1		 	  	  	  	 	  	
2								
3								
4								
5								
6								
7								
8								



Orden:

1) W

2) N

3) Q

4) S

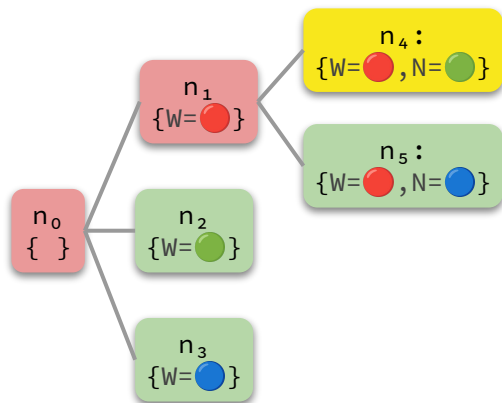
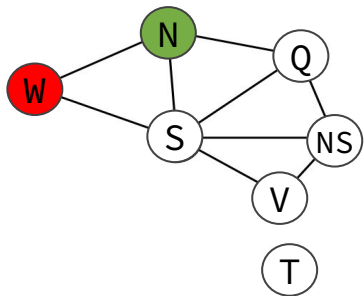
5) NS

6) V

7) T



Ejemplo



N° it.	Dominio						
	W	N	Q	NS	V	S	T
0	red, green, blue	red, green, blue	red, green, blue	red, green, blue	red, green, blue	red, green, blue	red, green, blue
1	red	green, blue	red, green, blue	red, green, blue	red, green, blue	green, blue	red, green, blue
2	red	green	red, blue	red, green, blue	red, green, blue	blue	red, green, blue
3							
4							
5							
6							
7							
8							

Comprobación hacia adelante.

Luego de asignar $N=\text{green}$, en las variables no asignadas S y Q , ambas relacionadas con N , se debe eliminar a green de su dominio.

Orden:

1) W

2) N

3) Q

4) S

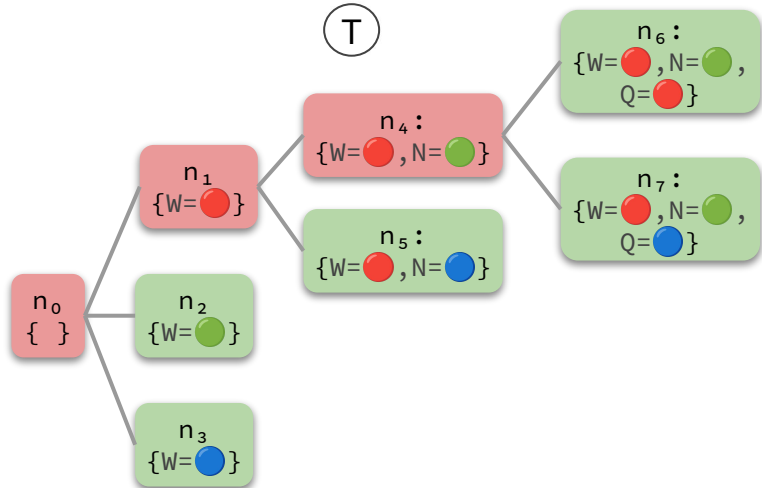
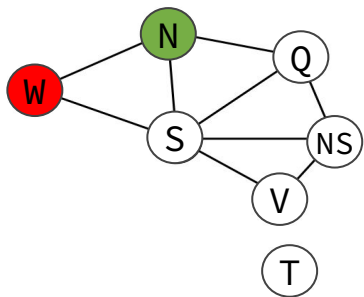
5) NS

6) V

7) T



Ejemplo



N° it.	Dominio						
	W	N	Q	NS	V	S	T
0							
1							
2							
3							
4							
5							
6							
7							
8							

Orden:

1) W

2) N

3) Q

4) S

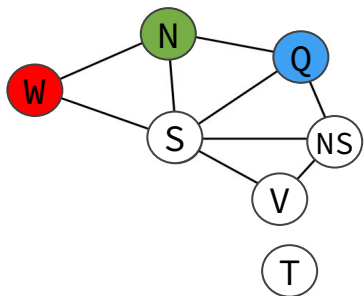
5) NS

6) V

7) T



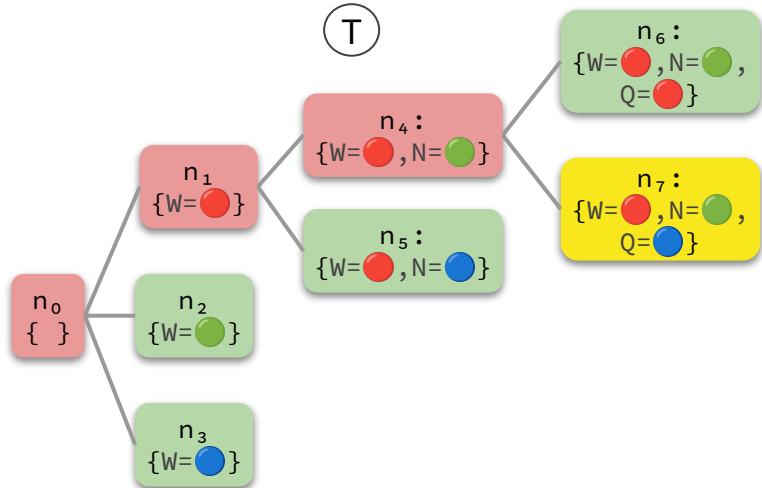
Ejemplo



N° it.	Dominio						
	W	N	Q	NS	V	S	T
0							
1							
2							
3						-	
4							
5							
6							
7							
8							

Comprobación hacia adelante.

Luego de asignar $Q = \text{blue}$, en las variables no asignadas S y NS , ambas relacionadas con Q , se debe eliminar a blue de su dominio.



Orden:

1) W

2) N

3) Q

4) S

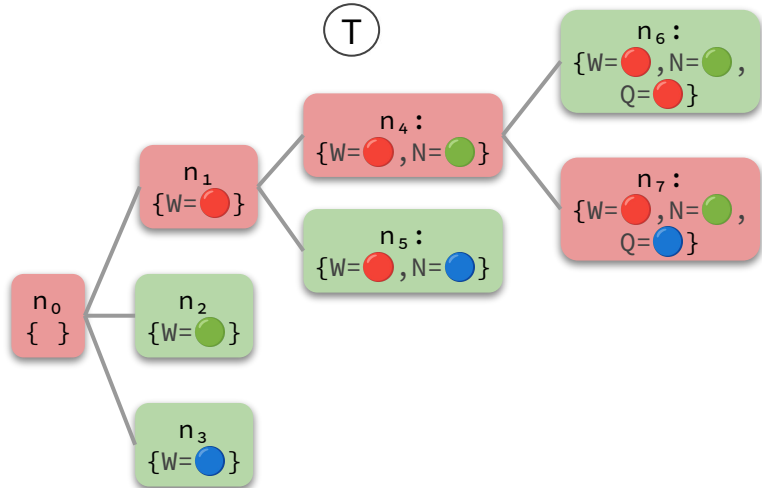
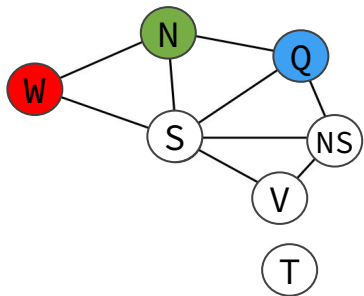
5) NS

6) V

7) T



Ejemplo



N° it.	Dominio						
	W	N	Q	NS	V	S	T
0							
1							
2							
3						-	
4							
5							
6							
7							
8							

El dominio de S quedó vacío.
**La comprobación hacia adelante
permite detectar que la
asignación es inconsistente.**
Backtracking debe retroceder.

Orden:

1) W

2) N

3) Q

4) S

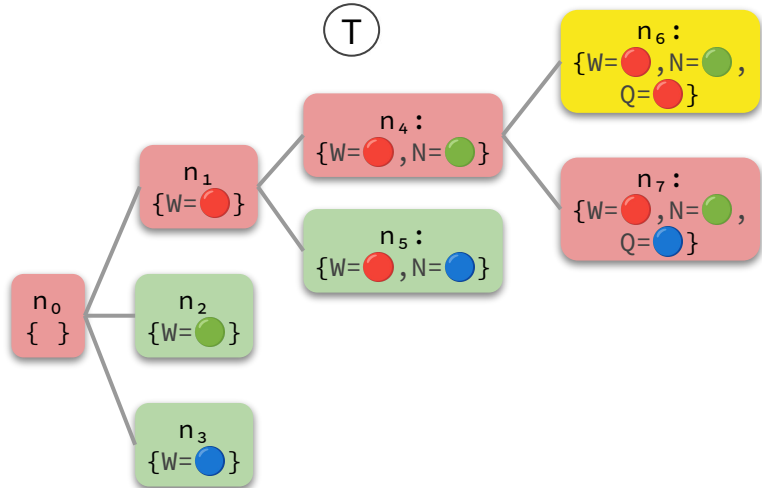
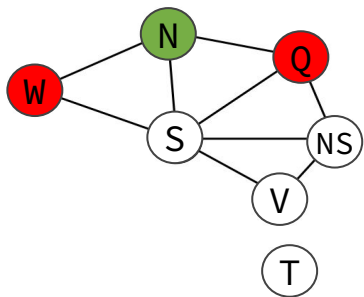
5) NS

6) V

7) T



Ejemplo



Orden:

1) W

2) N

3) Q

4) S

5) NS

6) V

7) T

N° it.	Dominio						
	W	N	Q	NS	V	S	T
0							
1							
2							
3							
4							
5							
6							
7							
8							

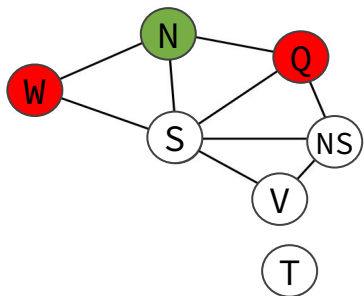
Comprobación hacia adelante.

Luego de asignar $Q = \text{red}$, en las variables no asignadas S y NS , ambas relacionadas con Q , se debe eliminar a red de su dominio.

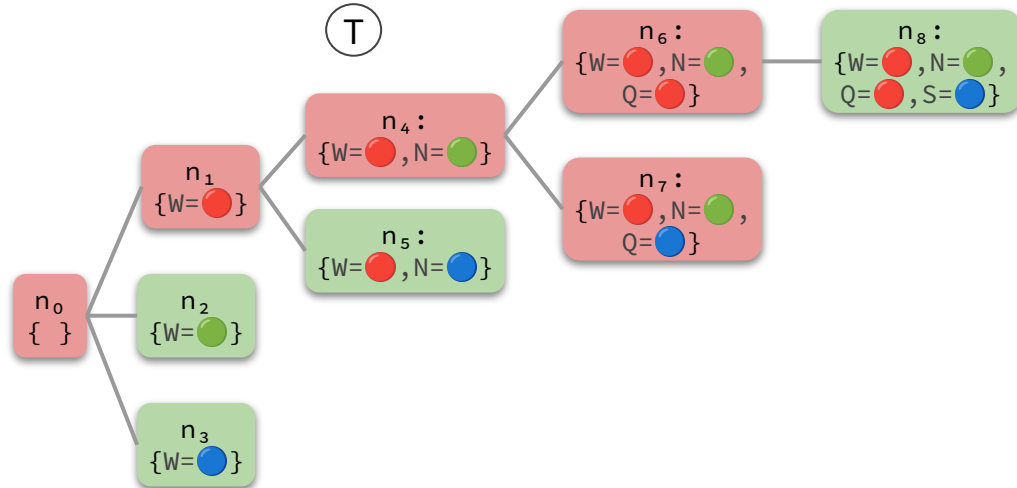
Notar que red había sido removido previamente del dominio de S (cuando se asignó a W), por lo tanto su dominio no se modifica.



Ejemplo



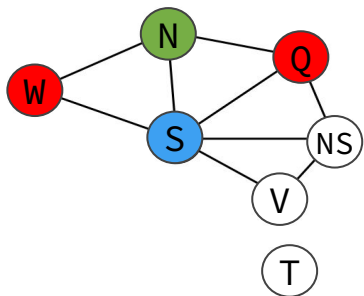
N° it.	Dominio						
	W	N	Q	NS	V	S	T
0							
1							
2							
3						-	
4							
5							
6							
7							
8							



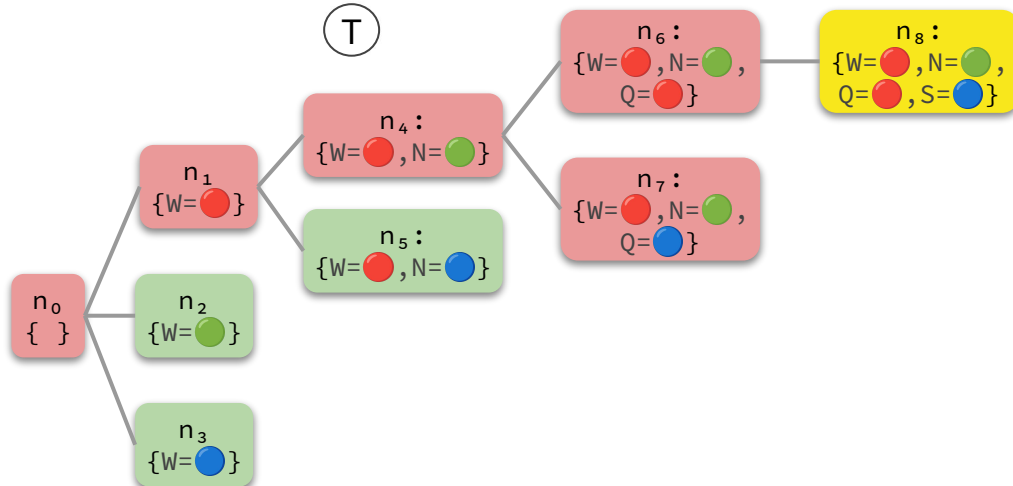
Orden: 1) W 2) N 3) Q 4) S 5) NS 6) V 7) T



Ejemplo



N° it.	Dominio						
	W	N	Q	NS	V	S	T
0							
1							
2							
3							
4							
5							
6							
7							
8							



Comprobación hacia adelante.

Luego de asignar $S = \text{blue}$, en las variables no asignadas V y NS , ambas relacionadas con S , se debe eliminar a blue de su dominio.

Orden:

1) W

2) N

3) Q

4) S

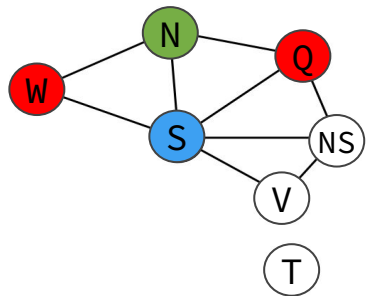
5) NS

6) V

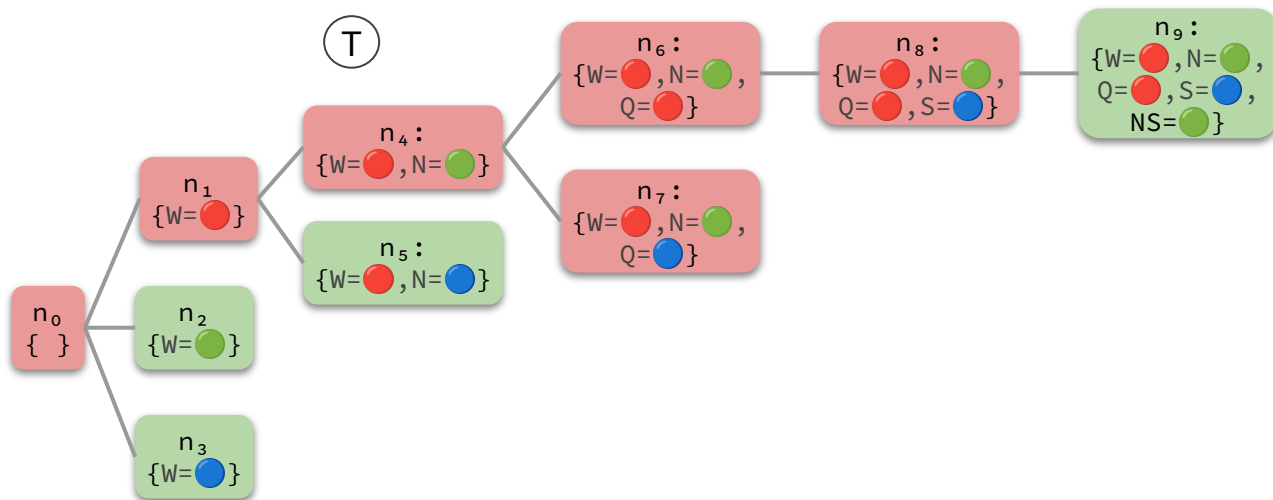
7) T



Ejemplo



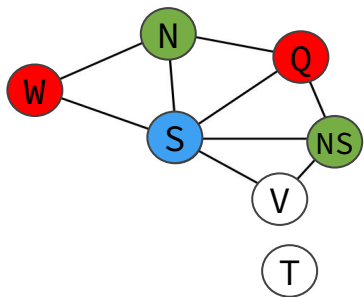
N° it.	Dominio						
	W	N	Q	NS	V	S	T
0							
1							
2							
3							
4							
5							
6							
7							
8							



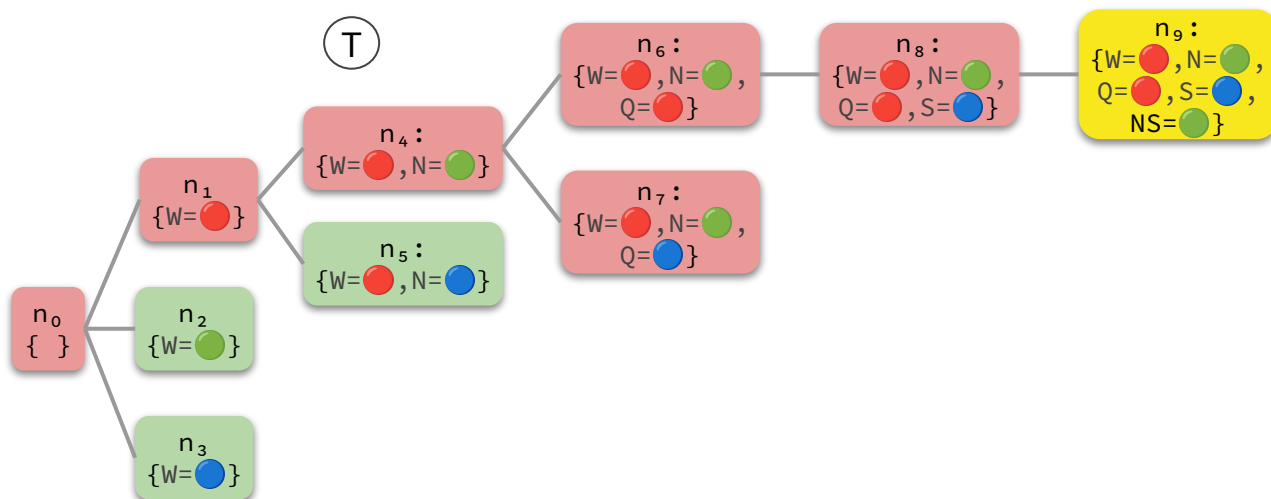
Orden: 1) W 2) N 3) Q 4) S 5) NS 6) V 7) T



Ejemplo



N° it.	Dominio						
	W	N	Q	NS	V	S	T
0							
1							
2							
3							
4							
5							
6							
7							
8							



Comprobación hacia adelante. Luego de asignar $NS=\text{green}$, en la variable no asignada V , relacionada con NS , se debe eliminar a de su dominio.

Orden:

1) W

2) N

3) Q

4) S

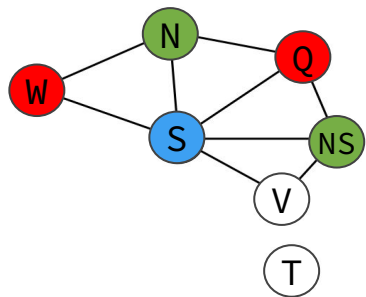
5) NS

6) V

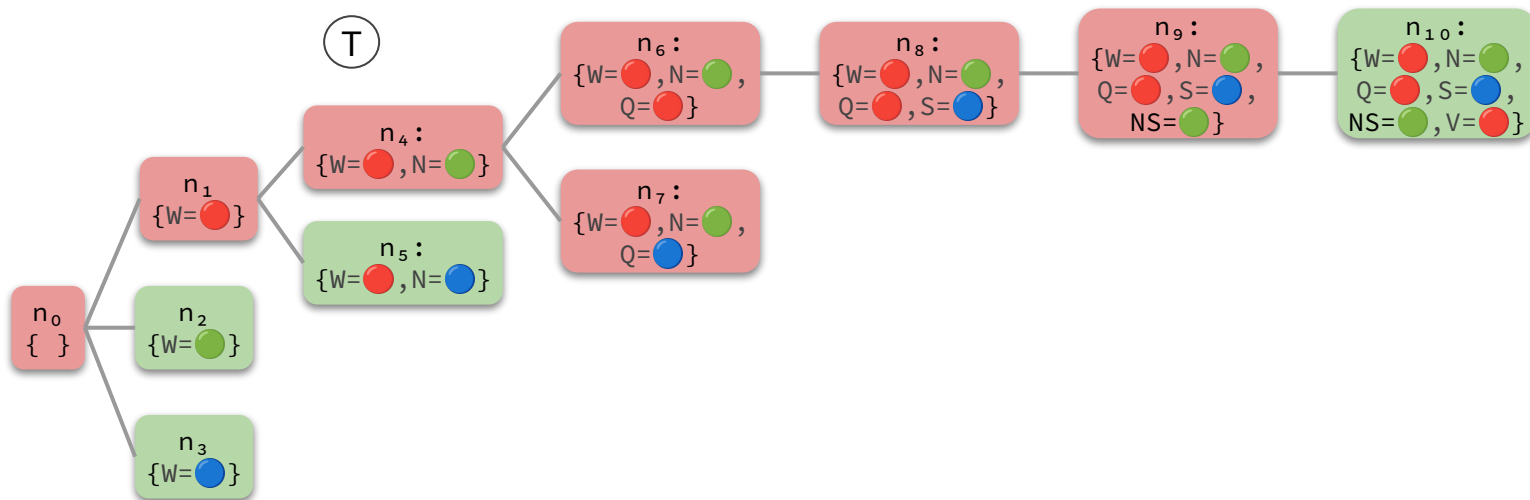
7) T



Ejemplo



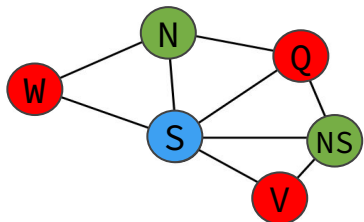
N° it.	Dominio						
	W	N	Q	NS	V	S	T
0							
1							
2							
3							
4							
5							
6							
7							
8							



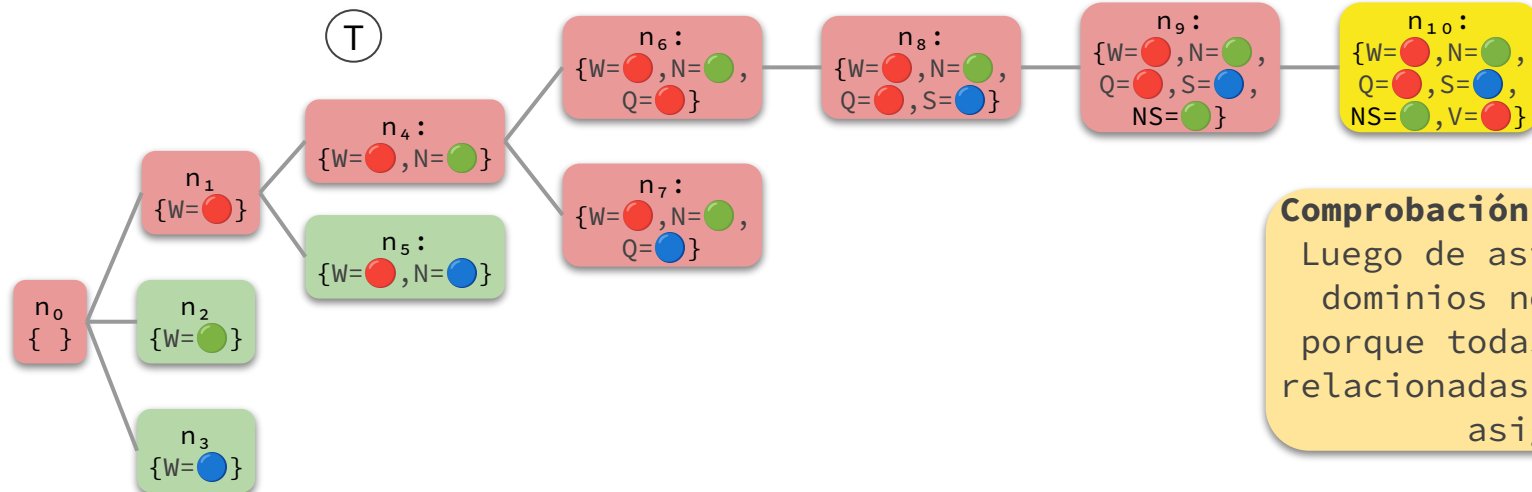
Orden: 1) W 2) N 3) Q 4) S 5) NS 6) V 7) T



Ejemplo



N° it.	Dominio						
	W	N	Q	NS	V	S	T
0							
1							
2							
3							
4							
5							
6							
7							
8							



Comprobación hacia adelante.

Luego de asignar $V=\text{red}$, los dominios no se modifican porque todas las variables relacionadas con V ya fueron asignadas.

Orden:

1) W

2) N

3) Q

4) S

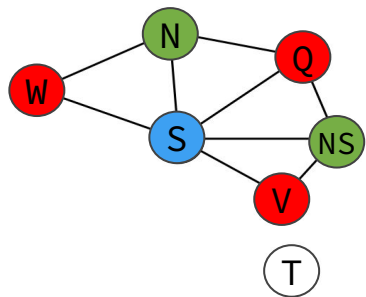
5) NS

6) V

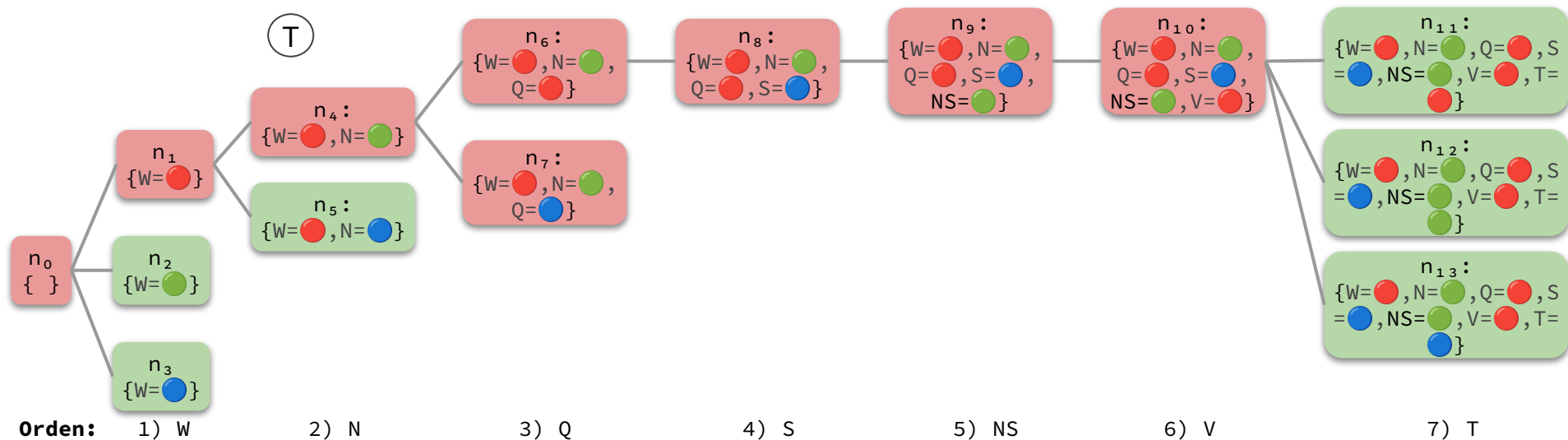
7) T



Ejemplo

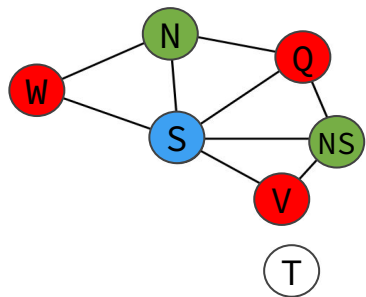


N° it.	Dominio						
	W	N	Q	NS	V	S	T
0							
1							
2							
3							
4							
5							
6							
7							
8							

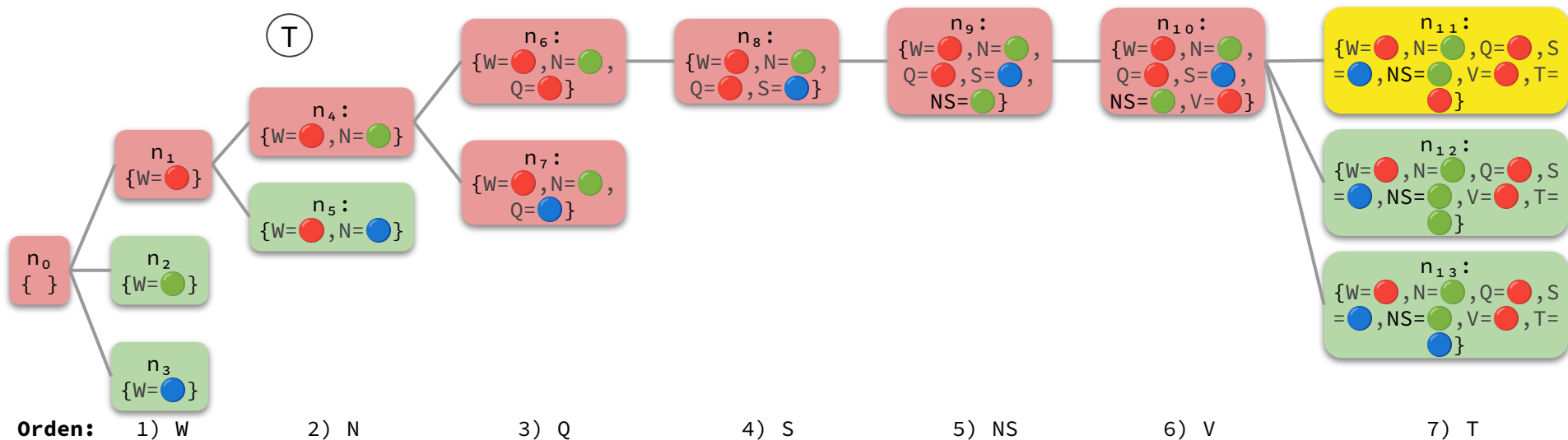




Ejemplo



N° it.	Dominio						
	W	N	Q	NS	V	S	T
0							
1							
2							
3							
4							
5							
6							
7							
8							



Ventajas

— — —

La comprobación hacia adelante...

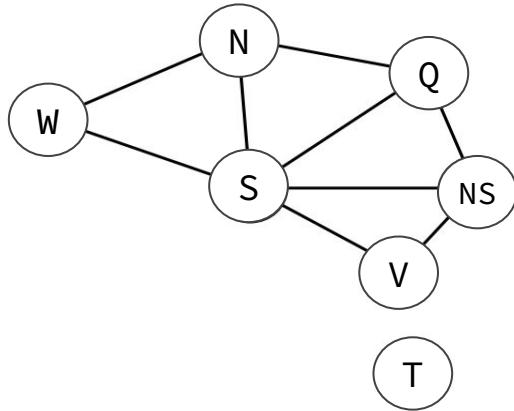
- es fácil de incorporar.
- reduce el **tamaño de los dominios** y por ende el **consumo de tiempo** de backtracking (ver línea 4 del pseudocódigo).
- simplifica el cómputo de **MVR**.
- detecta anticipadamente **inconsistencias** (algunas).

Veamos sus limitaciones...



Ejemplo – Coloreo de Australia

— — —

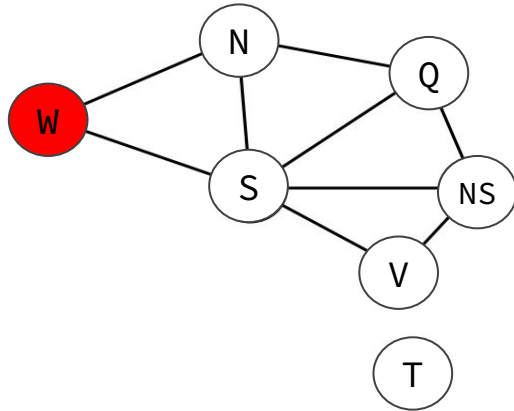


Acción	Dominios							
	W	N	Q	NS	V	S	T	
Inicio								



Ejemplo – Coloreo de Australia

— — —



Comprobación hacia adelante.

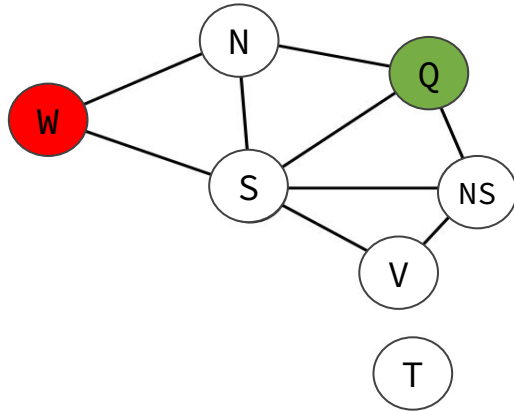
Luego de asignar $W = \text{red}$, se elimina a red de los dominios de N y S.

Acción	Dominios						
	W	N	Q	NS	V	S	T
Inicio	red green blue	red green blue	red green blue	red green blue	red green blue	red green blue	red green blue
(W, red)	red	green blue	red green blue	red green blue	red green blue	green blue	red green blue



Ejemplo – Coloreo de Australia

— — —



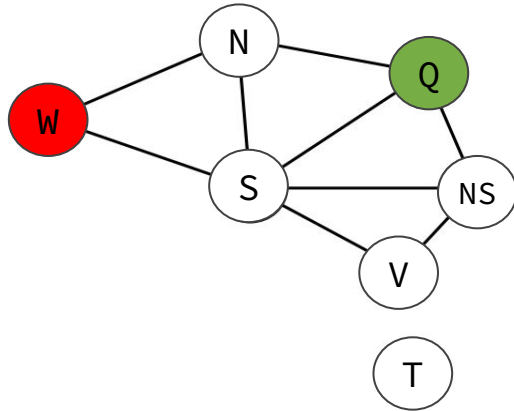
Comprobación hacia adelante.

Luego de asignar $Q = \text{green}$, se elimina a green de los dominios de N, S y NS.

Acción	Dominios							
	W	N	Q	NS	V	S	T	
Inicio								
(W,)								
(Q,)								

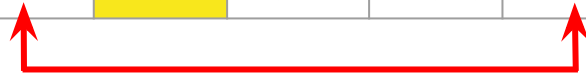


Ejemplo – Coloreo de Australia



Ningún dominio es vacío, pero la asignación actual $\{W=\text{red}, Q=\text{green}\}$ conduce inevitablemente a una **inconsistencia**, ya que N y S no pueden pintarse ambos de blue.

Acción	Dominios						
	W	N	Q	NS	V	S	T
Inicio	red green blue	red green blue	red green blue	red green blue	red green blue	red green blue	red green blue
(W, red)	red	green blue	red green blue	red green blue	red green blue	green blue	red green blue
(Q, green)	red	blue	green	red blue	red green blue	blue	red green blue



Limitaciones

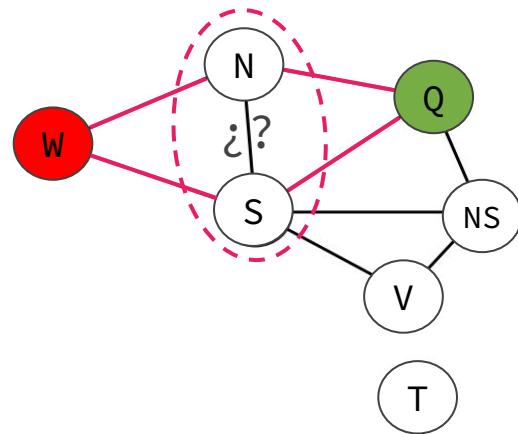
— — —

- La comprobación hacia adelante sólo restringe los dominios de las variables no asignadas que se relacionan con la variable asignada.

📖 **Ejemplo.** Al asignar $W = \text{red}$, se restringen los dominios de N y S para que sus valores sean consistentes con las restricciones $W \neq N$ y $W \neq S$.

- Pero **no mira lo suficientemente adelante.**

📖 **Ejemplo.** Para detectar la inconsistencia sería necesario restringir los dominios de N y S para que sean consistentes con $N \neq S$, es decir, entre dos variables no asignadas. ¿Cómo se hace? 🤔



Arco consistencia

Definición. Sean dos variables X_i y X_j y una restricción binaria $\langle (X_i, X_j), R \rangle$, donde R es una relación entre las variables X_i y X_j .

Decimos que X_i es **arco-consistente** con X_j si

$$\forall v_i \in D_i, \exists v_j \in D_j \text{ tal que } (v_i, v_j) \in R$$

o equivalentemente,

$$D_i \subseteq \text{dom}(R).$$

Es decir, si todo valor del dominio de la primer variable está relacionado con algún valor en el dominio de la segunda.

Ejemplo

— — —

Sea un CSP con dos variables X e Y con dominio $\{0,1,2,3,4,5,6,7,8,9\}$ y la restricción binaria:

$$\langle (X,Y), 5.X < Y \rangle$$

 ¿Es X arco consistente con Y ? 

Ejemplo

— — —

$$D_X = D_Y = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

Notamos con R a la relación dada por $5.X < Y$.

Para saber si X es arco-consistente con Y , tenemos que verificar si todo valor en el dominio de X está relacionado con algún valor en el dominio de Y , es decir, si para cada $v_X \in D_X$ existe $v_Y \in D_Y$ tal que $5.v_X < v_Y$.

Nos ayudaremos con una tabla.



Ejemplo

— — —

$$D_X = D_Y = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

Para cada posible valor de X,
completamos con todos los
posibles valores para Y que
verifiquen la relación

$$5. X < Y$$

X	Y
0	?
1	
2	
3	
4	
5	
6	
7	
8	
9	



Ejemplo

— — —

$$D_X = D_Y = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

Para cada posible valor de X ,
completamos con todos los
posibles valores para Y que
verifiquen la relación

$$5.X < Y$$

X	Y
0	1,2,3,4,5,6,7,8,9
1	?
2	
3	
4	
5	
6	
7	
8	
9	



Ejemplo

— — —

$$D_X = D_Y = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

Para cada posible valor de X,
completamos con todos los
posibles valores para Y que
verifiquen la relación

$$5. X < Y$$

X	Y
0	1,2,3,4,5,6,7,8,9
1	6,7,8,9
2	?
3	
4	
5	
6	
7	
8	
9	



Ejemplo

— — —

$$D_X = D_Y = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

Para cada posible valor de X,
completamos con todos los
posibles valores para Y que
verifiquen la relación

$$5. X < Y$$

X	Y
0	1,2,3,4,5,6,7,8,9
1	6,7,8,9
2	-
3	
4	
5	
6	
7	
8	
9	



Ejemplo

— — —

$$D_X = D_Y = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

Para cada posible valor de X,
completamos con todos los
posibles valores para Y que
verifiquen la relación

$$5. X < Y$$

X	Y
0	1,2,3,4,5,6,7,8,9
1	6,7,8,9
2	-
3	-
4	-
5	-
6	-
7	-
8	-
9	-



Ejemplo

— — —

De la tabla vemos que R por extensión es igual a $\{(0,1), (0,2), (0,3), (0,4), (0,5), (0,6), (0,7), (0,8), (0,9), (1,6), (1,7), (1,8), (1,9)\}$.

Además, $\text{dom}(R) = \{0,1\}$.

X	Y
0	1,2,3,4,5,6,7,8,9
1	6,7,8,9
2	-
3	-
4	-
5	-
6	-
7	-
8	-
9	-



Ejemplo

— — —

De la tabla vemos que R por extensión es igual a $\{(0,1), (0,2), (0,3), (0,4), (0,5), (0,6), (0,7), (0,8), (0,9), (1,6), (1,7), (1,8), (1,9)\}$.

Además, $\text{dom}(R) = \{0,1\}$.

Como existen valores de X que no están relacionados con ningún valor de Y , por ejemplo $X = 2$, resulta que **X no es arco-consistente con Y .**

Otra forma de justificarlo es diciendo que $D_x = \{0,1,2,3,4,5,6,7,8,9\}$ no está contenido en $\text{dom}(R) = \{0,1\}$.

X	Y
0	1,2,3,4,5,6,7,8,9
1	6,7,8,9
2	-
3	-
4	-
5	-
6	-
7	-
8	-
9	-

Forzar arco-consistencia

— — —

- Es posible **forzar arco-consistencia** de una variable X_i con una variable X_j .
- La idea es eliminar todos los valores del dominio de X_i que violen la condición de arco-consistencia.



Ejemplo (continuación)

— — —

De la tabla vemos que R por extensión es igual a $\{(0,1), (0,2), (0,3), (0,4), (0,5), (0,6), (0,7), (0,8), (0,9), (1,6), (1,7), (1,8), (1,9)\}$.

Además, $\text{dom}(R) = \{0,1\}$.

Restringiendo D_x a $\text{dom}(R) = \{0,1\}$, resulta X arco-consistente con Y .

X	Y
0	1,2,3,4,5,6,7,8,9
1	6,7,8,9

Arco consistencia y comprobación hacia adelante

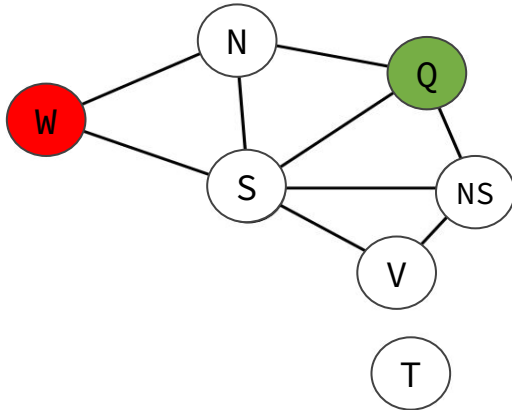
Es posible redefinir la comprobación hacia adelante usando la noción de arco-consistencia (es equivalente a la definición que vimos antes)

- **Comprobación hacia adelante** (alternativa). Siempre que se asigne una variable X_i , para cada variable no asignada X_j que esté relacionada con X_i , se fuerza que X_j sea **arco-consistente** con X_i .



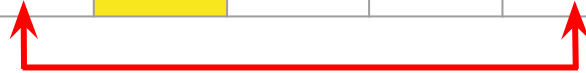
Ejemplo – Coloreo de Australia

Volvamos al ejemplo anterior.



La asignación actual $\{W=\text{red}, Q=\text{green}\}$ conduce a una **inconsistencia**, pero la comprobación hacia adelante no la detecta porque ningún dominio está vacío.

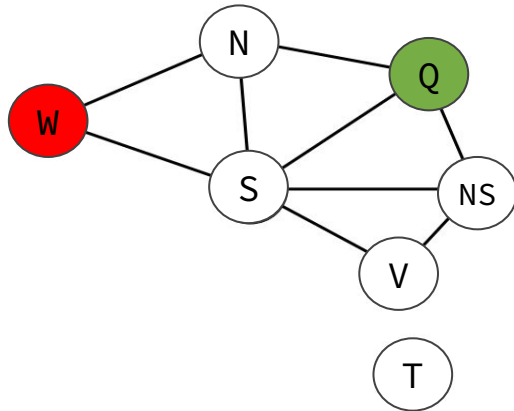
Acción	Dominios						
	W	N	Q	NS	V	S	T
Inicio	red green blue	red green blue	red green blue	red green blue	red green blue	red green blue	red green blue
(W, red)	red	green blue	red green blue	red green blue	red green blue	green blue	red green blue
(Q, green)	red	blue	green	red blue	red green blue	blue	red green blue





Ejemplo – Coloreo de Australia

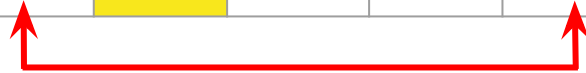
Volvamos al ejemplo anterior.



¿Y si forzamos que N sea arco-consistente con S, es decir, **entre dos variables no asignadas**? Veamos... 🤔

La asignación actual $\{W=\text{red}, Q=\text{green}\}$ conduce a una **inconsistencia**, pero la comprobación hacia adelante no la detecta porque ningún dominio está vacío.

Acción	Dominios						
	W	N	Q	NS	V	S	T
Inicio	red green blue	red green blue	red green blue	red green blue	red green blue	red green blue	red green blue
(W, red)	red	green blue	red green blue	red green blue	red green blue	green blue	red green blue
(Q, green)	red	blue	green	red blue	red green blue	blue	red green blue





Ejemplo – Coloreo de Australia

— — —

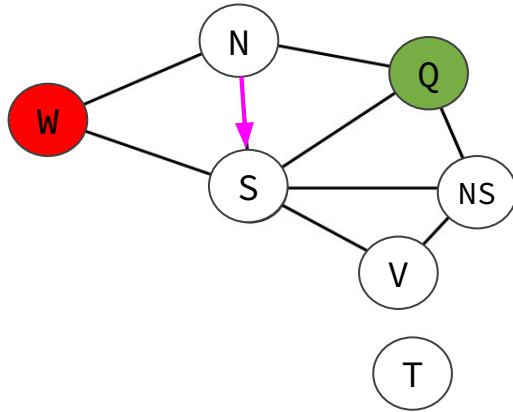
Acción	Dominios						
	W	N	Q	NS	V	S	T
Inicio	  	  	  	  	  	  	  
(W, )		 	  	  	  	 	  
(Q, )				 	  		  

$D_N = D_S = \{\text{blue dot}\}$ y las variables N y S están relacionadas por $N \neq S$.

Es claro que para la asignación $N = \text{blue dot}$ no existe un valor para S que sea distinto de blue dot . Por lo tanto, **N no es arco-consistente con S.**



Ejemplo – Coloreo de Australia



Para que N sea arco consistente con S...

...removemos a  del dominio de N.

Acción	Dominios						
	W	N	Q	NS	V	S	T
Inicio	  	  	  	  	  	  	  
(W, )		 	  	  	  	 	  
(Q, )				 	  		  
		-			 		  

El dominio de N quedó vacío.

Detectamos anticipadamente que la asignación es **inconsistente**.

4. Propagación de restricciones

— — —

- La **comprobación hacia adelante** y la **arco-consistencia** realizan un tipo de **inferencia** llamada **propagación de restricciones**: usar las restricciones para descartar valores inconsistentes para las variables.
- Restringir el dominio de una variable puede permitir restringir el dominio de otras, y así en **cadena**.
- La comprobación hacia adelante solo fuerza arco-consistencia con la variable asignada, pero no sigue propagando en cadena.

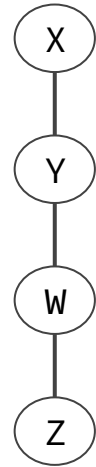


Ejemplo – Propagación de restricciones en cadena

— — —

Sea un CSP con variables X , Y , W y Z , todas con dominio $\{1,2,3,4\}$, y las restricciones $X < Y$, $Y < W$ y $W < Z$. Supongamos que se asigna $X = 2$.

1. La comprobación hacia adelante fuerza que Y sea arco-consistente con X , restringiendo el dominio de Y a $\{3,4\}$.
2. Ahora podemos forzar que W sea arco-consistente con Y , restringiendo el dominio de W a $\{4\}$.
3. Ahora podemos forzar que Z sea arco-consistente con W , restringiendo el dominio de Z a $\{\}$. Hemos detectado que la asignación $X = 2$ conduce a una inconsistencia.



Grafo de restricciones.

4. Propagación de restricciones

— — —

- Existen algoritmos de propagación de restricciones estrictamente más potentes que la comprobación hacia adelante.
- **Mantenimiento de la consistencia de arco (MCA)**. Siempre que se asigne una variable, se impone (iterativamente) la arco-consistencia entre **todas** las variables hasta que no haya cambios en los dominios.
- El algoritmo usado para imponer arco consistencia se denomina **AC-3** (Mackworth, 1977) y utiliza una cola. No lo vamos a ver.

4. Propagación de restricciones

— — —

- MCA detecta anticipadamente **más inconsistencias** que la comprobación hacia adelante, pero es **considerablemente más costoso**.
- En algunos problemas, MCA puede mejorar el **consumo de tiempo**, en otros puede empeorar debido a la **sobrecarga computacional**.

5. Restricciones globales

— — —

Las restricciones globales permiten detectar inconsistencias de manera más eficiente que los métodos generales presentados hasta ahora.

- La restricción

$$\langle (X_1, \dots, X_m), \textit{Alldiff}(X_1, \dots, X_m) \rangle$$

representa que las variables $X_1, \dots, X_m$ deben tener valores distintos.

- La restricción

$$\langle (X_1, \dots, X_m), \textit{Atmost}(K, X_1, \dots, X_m) \rangle$$

representa que los valores de $X_1, \dots, X_m$ suman a lo sumo K , siendo K alguna constante numérica. Es decir, es equivalente a:

$$\langle (X_1, \dots, X_m), X_1 + \dots + X_m \leq K \rangle$$



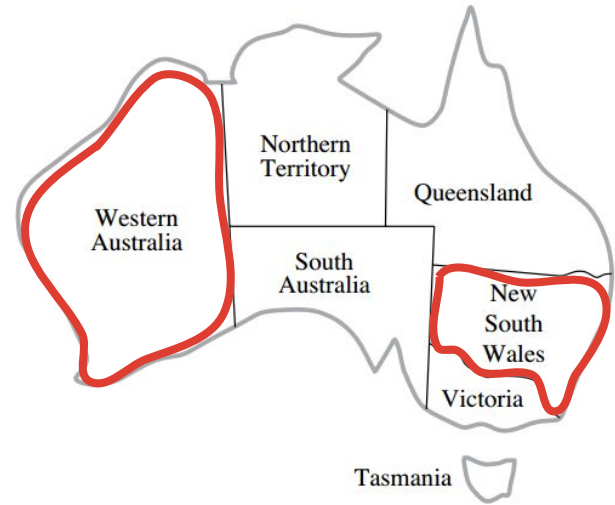
Ejemplo – Coloreo de Australia

Considerar la asignación parcial $\{W=\text{red}, NS=\text{red}\}$.
Por comprobación hacia adelante, los dominios de N, Q, V y S son iguales a $\{\text{green}, \text{blue}\}$.

Notar que las regiones Northern Territory, Queensland y South Australia son mutuamente fronterizas, es decir, vale la restricción:

$$\langle (N, Q, S), \text{Alldiff}(N, Q, S) \rangle$$

Dado que hay 3 variables que necesitan valores distintos y solo 2 colores en sus dominios, podemos inferir que hay una inconsistencia.



Restricciones globales – *Alldiff*

— — —

$$\langle (X_1, \dots, X_m), \textit{Alldiff}(X_1, \dots, X_m) \rangle$$

Test de inconsistencia. Si en los dominios hay n posibles valores distintos para $X_1, \dots, X_m$ y $n < m$, entonces la restricción no se puede verificar.



Ejemplo

— — —

Consideremos un CSP con tres variables X , Y y Z con dominio $\{5,6,7,8,9\}$, relacionados por medio de la siguiente restricción:

$$\langle (X,Y,Z), \textit{Atmost}(14,X,Y,Z) \rangle$$

De los dominios, sabemos que el menor valor que puede asumir cada una de las variables es 5. Es decir, podemos inferir que las variables verifican la desigualdad

$$X + Y + Z \geq 15$$

Luego, la desigualdad

$$X + Y + Z \leq 14$$

no se puede verificar y el CSP no tiene solución.

Restricciones globales – *Atmost*

— — —

$$\langle (X_1, \dots, X_m), \textbf{Atmost}(K, X_1, \dots, X_m) \rangle$$

- **Test de inconsistencia.** Si la suma del menor valor del dominio de cada una de las variables $X_1, \dots, X_m$ es mayor a K , entonces la restricción no se puede verificar.

Estructura de los problemas

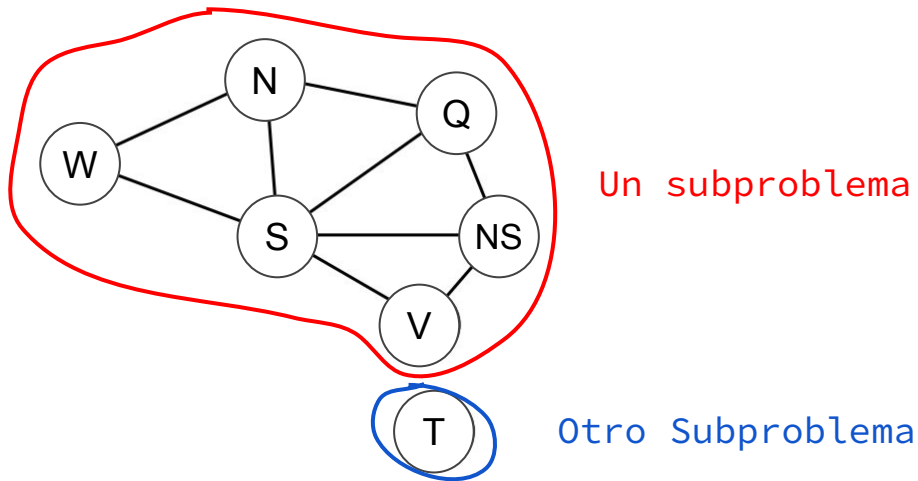
La dificultad para resolver un CSP está fuertemente relacionada con la **estructura** del grafo de restricciones.

Una de las estructuras más importantes son las **componentes conexas** del grafo.

— — —

Ejemplo – Coloreo de Australia

Colorear Tasmania y colorear el continente son **subproblemas independientes** – cualquier solución para Tasmania combinada con cualquier solución para el continente conduce a una solución para el mapa completo.



Ventajas

Suponer que un CSP tiene n variables con d posibles valores y que se divide en k subproblemas independientes, cada uno de ellos con n/k variables.

La cantidad de asignaciones completas en el problema original es:

$$d^n$$

La cantidad de asignaciones completas en los subproblemas es:

$$k \cdot d^{n/k}$$

 **Ejemplo.** Para $n=80$, $d=2$ y $k=4$, pasamos de $1,2 \times 10^{24}$ a 4.194.304 asignaciones completas.

Teoría de grafos

La **teoría de grafos** se ocupa de estudiar la estructura de los grafos, en particular de los grafos de restricciones que representan a un CSP.

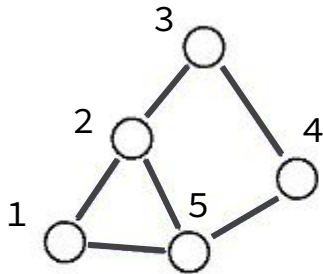
Para definir formalmente qué son las componentes conexas, debemos repasar algunos conceptos de teoría de grafos.

Subgrafo inducido

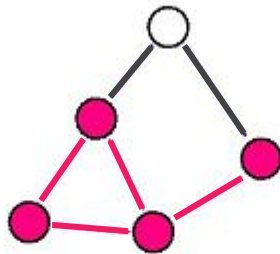
Dado un grafo $G=(V,E)$ y un subconjunto $U \subseteq V$, se define el **subgrafo inducido** $G[U]$ como aquél que tiene a U como conjunto de vértices y toda arista de G con extremos en U es una arista de $G[U]$.



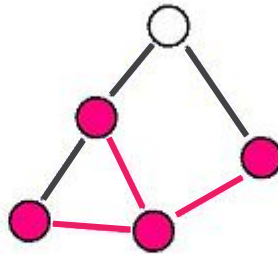
Ejemplos.



Grafo original



Subgrafo inducido por $\{1,2,5,4\}$.



No es subgrafo inducido porque falta la arista $(1,2)$.

Componente conexa

— — —

- Una **componente conexa** de un grafo es un subgrafo inducido que es:
 - **Conexo**. Todo par de vértices están conectados por un camino.
 - **Maximal**. Se desconecta al agregar cualquier otro vértice del grafo.
- **Cada componente conexa del grafo se corresponde con un subproblema independiente.**
- Existen algoritmos para encontrar las componentes conexas de un grafo, que están basados en DFS: algoritmo de Kosaraju, algoritmo de Tarjan, entre otros.



Próximamente

— — —

Volveremos a hablar de algoritmos de búsqueda local. Nos ocuparemos de una familia de algoritmos que han demostrado ser muy efectivos y que tienen la particularidad de estar inspirados en la **naturaleza** 🍁.